

PPP 2025/2026 MPI Assignment - N-body application

Kees Verstoep

December 2025

Contents

1 Introduction **2**

2 Sequential Implementation **4**

3 Parallel MPI Implementation **6**
3.1 The MPI Message-Passing Interface 6
3.2 Parallelizing N-body with MPI 7
3.2.1 Base N-body parallelization strategy 7
3.2.2 Extended parallelization: Load balance improvement . . . 8
3.2.3 Extended parallelization: Gather positions 8
3.3 Running your MPI implementation 9
3.4 Correctness 9
3.5 Performance evaluation 10
3.6 Report 10

4 Assignment **11**

5 Contact **12**

1 Introduction

N-body Simulation

The N-body problem concerns computing the trajectory of objects in space as driven by gravitational forces. You can find more details about the problem here: `https://en.wikipedia.org/wiki/N-body_simulation`
For a 3-dimensional N-body simulation, the force components for a body are calculated by summing the gravitational forces from all other bodies in the system. The unit vector is used to establish the correct direction for each individual force before they are summed. Finally, the total force can be used with a numerical integration, like the explicit Euler method (which we use here), to update the body's position and velocity.

Calculating the force components

Let's consider a system of N bodies. We want to find the total force on body i, with mass m_i and position vector $\vec{r_i}$. The total force $\vec{F_i}$ on body i is the vector sum of the gravitational forces exerted on it by every other body j:

$$\vec{F_i} = \sum_{j=1, j\neq i}^{N} \vec{F_{ij}}$$

where $\vec{F_i}$ is the force exterted on body i by body j.
The individual force vector $\vec{F_i}$ is defined by Newton's Law of Universal Gravitation:

$$\vec{F_{ij}} = \frac{G m_i m_j}{r_{ij}^2} \hat{r}_{ij}$$

where:

- G is the gravitational constant.
- m_j is the mass of body j.
- r_{ij} is the scalar distance between body i and body j.
- $\hat{r}_{ij}$ is the unit vector pointing from body i to body j.

To get the 3D components of $\vec{F_{ij}}$ we follow these steps:

1. Calculate the distance vector

First, find the vector $\vec{r}_{ij}$ that points from body i to body j:

$$\vec{r}_{ij} = \vec{r}_j - \vec{r}_i = (x_j - x_i)\hat{i} + (y_j - y_i)\hat{j} + (z_j - z_i)\hat{k}$$

2. Calculate the scalar distance

Next, find the magnitude of the distance vector r_{ij} using the Pythagorean theorem for 3D vectors:

$$r_{ij} = |\vec{r}_{ij}| = \sqrt{(x_j - x_i)^2 + (y_j - y_i)^2 + (z_j - z_i)^2}$$

3. Determine the unit vector

The unit vector $\hat{r}_{ij}$ is found by dividing the distance vector by its magnitude. This gives a vector of length 1 that points in the directorion of the force:

$$\hat{r}_{ij} = \frac{\vec{r}_{ij}}{|\vec{r}_{ij}|} = \frac{1}{r_{ij}}(r_{ij,x}\hat{i} + r_{ij,y}\hat{j} + r_{ij,z}\hat{k})$$

where $\hat{i} = (1, 0, 0)$, $\hat{j} = (0, 1, 0)$, and $\hat{k} = (0, 0, 1)$.

4. Calculate the force components

For the force components, substitute the unit vector and scalar distance back into the force equation for $\vec{F}_{ij}$:

$$\vec{F}_{ij} = \frac{Gm_i m_j}{r_{ij}^2} \frac{\vec{r}_{ij}}{r_{ij}}$$

N-body calculation with the Euler method

In a simulation, the total force and acceleration for each body are calculated iteratively. The simplest numerical integration technique is the explicit Euler method, which we will use here.

Simulation loop (at each time step Δt)

1. Initialize total force

For each body i, set its total force vector $F_i = (0, 0, 0)$.

2. Calculate cumulative force

For each pair of bodies (i, j) where $i \neq j$:

1. Calculate the distance vector $\vec{r}_{ij} = \vec{r}_j - \vec{r}_i$
2. Calculate the scalar distance $r_{ij} = |\vec{r}_{ij}|$
3. Calculate the pairwise force vector $\vec{F}_{ij} = \frac{Gm_i m_j}{r_{ij}^3}|\vec{r}_{ij}|$
4. Add this force to the total for both bodies (by Newton's Third Law, $\vec{F}_{ji} = -\vec{F}_{ij}$), so: $\vec{F}_i = \vec{F}_i + \vec{F}_{ij}$ and $\vec{F}_j = \vec{F}_j - \vec{F}_{ij}$

```
# Vector of three components
struct Vec
    x::Float64
    y::Float64
    z::Float64
end

# Data for ALL bodies
struct Bodies
    pos::Vector{Vec}
    force::Vector{Vec}
    vel::Vector{Vec}
    mass::Vector{Float64}
end
```

Figure 1: The `Bodies` structure.

3. Update positions and velocities (using explicit Euler)

For each body i:

1. Update the position: $\vec{r}_{i,new} = \vec{r}_{i,old} + \vec{v}_{i,old}.\Delta t$
2. Calculate the acceleration: $\vec{a}_i = \frac{\vec{F}_i}{m_i}$
3. Update the velocity: $\vec{v}_{i,new} = \vec{v}_{i,old} + \vec{a}_i.\Delta t$

The process is then repeated for the next time step. The explicit Euler method is a simple first order integrator, and more accurate and numerically stable methods ate often used in practice, but the advantage is that is easy to understand and suitable for parallellization.

2 Sequential Implementation

You should start by inspecting and understanding the sequential code we provided. Inside 'nbody-seq-src.jl', you'll see a loop in perf_nbody(). Executing 'compute_forces!()', 'compute_velocities!()', and 'compute_positions!()' is where you can get parallel execution.
It may take a little rewriting, but the idea is to have the work of updating the various body forces, velocities, and positions be done in parallel. The forces are the most important to compute in parallel, but new positions and velocities based on this can in principle be computed without communication.
To run the sequential program locally on your laptop use:

```
$ julia -O3 nbody-seq-run.jl 32 1000000 1
```

To run the sequential program on a compute node of DAS-5 (to avoid running it on the headnode, which you should not do), you can use `prun -np 1`, so:

```
$ prun -np 1 julia -O3 nbody-seq-run.jl 32 1000000 1
```

```
function compute_forces!(bodies)
    clear_forces!(bodies)

    # Incrementally accumulate forces from each body pair,
    # skipping force of body on itself (c == b)
    @inbounds for b in 1:length(bodies.force)
        # Get data for body b
        mass_b = bodies.mass[b]
        pos_b = bodies.pos[b]
        force_b = bodies.force[b]

        # We have order N*N/2 force calculations
        # with N = length(bodies.force)
        @inbounds for c in b + 1:length(bodies.force)
            #Get data for body c
            pos_c = bodies.pos[c]
            mass_c = bodies.mass[c]
            force_c = bodies.force[c]

            # Distance
            dx = pos_c.x - pos_b.x
            dy = pos_c.y - pos_b.y
            dz = pos_c.z - pos_b.z
            dist_sqr = dx^2 + dy^2 + dz^2
            dist = sqrt(dist_sqr)

            # Force between b and c
            # Assume there are no singularities, resulting in division by zero
            # also, we use gravity constant 1
            coefficient = ((mass_b * mass_c) / (dist * dist_sqr))
            dfx = coefficient * dx
            dfy = coefficient * dy
            dfz = coefficient * dz

            # Add contribution to c
            fx = force_c.x - dfx
            fy = force_c.y - dfy
            fz = force_c.z - dfz
            bodies.force[c] = Vec(fx,fy,fz)

            # Update contribution to b
            fx = force_b.x + dfx
            fy = force_b.y + dfy
            fz = force_b.z + dfz
            force_b = Vec(fx,fy,fz)
        end

        # Add contribution to b
        bodies.force[b] = force_b
    end
end
```

Figure 2: The `compute_forces!()` function.

```
function compute_positions!(bodies, dt)
    @inbounds for b in 1:length(bodies.pos)
        pos_b = bodies.pos[b]
        vel_b = bodies.vel[b]
        x = pos_b.x + dt * vel_b.x
        y = pos_b.y + dt * vel_b.y
        z = pos_b.z + dt * vel_b.z
        bodies.pos[b] = Vec(x,y,z)
    end
end

function compute_velocities!(bodies, dt)
    @inbounds for b in 1:length(bodies.vel)
        mass_b = bodies.mass[b]
        force_b = bodies.force[b]
        vel_b = bodies.vel[b]
        dt_div_mass_b = (dt / mass_b)
        vx = vel_b.x + dt_div_mass_b * force_b.x
        vy = vel_b.y + dt_div_mass_b * force_b.y
        vz = vel_b.z + dt_div_mass_b * force_b.z
        bodies.vel[b] = Vec(vx,vy,vz)
    end
end
```

Figure 3: The `compute_positions!()` and `compute_velocities!()` functions.

This means that there are 32 bodies generated with varying mass, initial positions and velocities, which are updated in the simulation for 1000000 iterations, and the experiment is only executed once. The application prints the relevant part of the total running time (excluding initialization) after which the resulting end positions and velocities are printed. Since this concerns a large number of floating point values, it makes sense to redirect the output to a file, so they can be compared for consistency.

3 Parallel MPI Implementation

Your task is to convert the sequential program in Julia provided into a parallel program using the MPI (Message Passing Interface) communication library. The application has to be run and evaluated on DAS-5.

3.1 The MPI Message-Passing Interface

The Message-Passing Interface MPI is widely used. It is not a new programming language; rather it is a library of subprograms that can be called from C, Fortran and Julia programs. It was developed by an open, international forum consisting of representatives from industry, academia, and government laboratories. It has rapidly received widespread acceptance because it has been carefully designed to permit maximum performance on a wide variety of systems, and it is based

```
function perf_nbodies(nbodies, iterations, verbose=false, silent=false)
    bodies = alloc_bodies(nbodies)
    init_bodies!(bodies)

    # don't include initialization time
    t1 = time()

    dt = 0.01
    for i in 1:iterations
        if verbose
            print_bodies(bodies, i)
        end
        compute_forces!(bodies)
        compute_positions!(bodies, dt)
        compute_velocities!(bodies, dt)
    end

    t2 = time()
    secs = t2 - t1
    @printf("nbody took %.3f secs\n", secs)

    if ! silent
        print_bodies(bodies, iterations+1)
    end
end
```

Figure 4: The `perf_nbodies()` function.

on message passing, one of the most powerful and widely used paradigms for programming parallel systems. The introduction of MPI makes it possible for developers of parallel software to write libraries of parallel programs that are both portable and efficient. Use of these libraries will hide many of the details of parallel programming, and, as a consequence, make parallel computing much more accessible to students and professionals in all branches of science and engineering.

3.2 Parallelizing N-body with MPI

There are several options to parallellize the N-body application using MPI. Looking at the main algorithm, the most computation-intensive function is `compute_forces!()` which in every iteration performs $O(N * N/2)$ force operations.

3.2.1 Base N-body parallelization strategy

Assuming your MPI application keeps a copy of every body state, it can spread the computation of all force pairs by suitably splitting this over the available MPI ranks and aggregating the partial results. This can be done by defininining a mapping of the body indexes to MPI ranks, and using this inside `compute_forces!()`. With the partial force components all computed, an

efficient way to sum them is by using a version of the collective `MPI.Allreduce` primitive.

3.2.2 Extended parallelization: Load balance improvement

The efficiency of the base MPI version will also depend on the details of the distribution of bodies over MPI ranks. Depending on the mapping, some MPI ranks may be computing less force pairs than others, thus causing load imbalance which could have consequences on efficiency. This can be analyzed in theory, but also implemented and evaluated in practice.
This first extension needs to be implemented in addition of the base version, and its performance needs to be compared with it. For this reason, do not fine-tune the body distribution in the base version, but leave that to this extension.

3.2.3 Extended parallelization: Gather positions

With all ranks having access to all aggregated forces, the base MPI version can simply compute all new positions at every rank. Although this is a simple approach, the position update computation could alternatively be spread over the MPI ranks, following the body distribution, and then synchronized using a version of `MPI.Allgather`. How efficient distrtibuted computation and gathering of position updates is, will depend on the trade-off between the duplicate computations avoided and the additional communication step required.
Note that it is not needed to globally synchronize all velocities at every time step, as they don't directly impact remote body parameters (they only need to be synchronized at the end when the full body states including velocities are printed).
The extension of gathering positions needs to be implemented *after* the base and load balanced versions. Only after this, you could work on analyzing the impact of position gathering.

Comparing multiple implementations

The implementations should all be part of a single parallel application, where the specific functionality is determined by run-time flags. Having all implementations integrated this way should help avoid code duplications and in practice reduce development time. This particular application is also quite suitable for this approach.
Your implementation, evaluation and analysis of the base MPI implementation (see Section 3.2.1) and first extended parallelization (see Section 3.2.2) can give you at most 8 points.
To make the parallel evaluation honest, leave the base sequential code unchanged as much as possible, and focus the MPI version on *minimal* changes to the application (e.g., don't change the base algorithms, code, or data structures). The default parallel version should be the base implementation, while the flag `-balance` should use the optimized load distribution.

To get the final two points for this assignment, you will also need to implemeent, evaluate and analyze the second extended version, as discussed in Section 3.2.3. Be sure to clearly describe your additional work in your report, explaining what is different about the additional version, and how this impacts performance. The application option `-gather` should enable the functionality related to position gathering, and avoid duplicate position computations. The extension should work both in combination with the base version as with the version using load balanced object distribution.

3.3 Running your MPI implementation

You are advised to use the `mpiexecjl` package to run your MPI application. After setting this up (see Canvas for additional documentation) this then works as follows on your laptop:

```
$ mpiexecjl -np <np> julia -O3 nbody-par-run.jl <nbody> <iter> <runs>
```

Here `<np>` is the number of MPI processes, `<nbody>` is the number of bodies, `<iter>` is the number of iterations of the algorithm, and `<runs>` is the number of times the experiment is repeated. The `<nbody>` argument may be preceeded with options like `-s` (silent run, not printing body details), `-v` (verbose run, printing body details in every iteration), or options like `-balance` and/or `-gather` as implemented besides your base version.
On DAS-5 you will need to run your MPI application on the compute nodes. The easiest way to do this for a Julia MPI application, mostly mirroring the laptop commandline, is to prefix your Julia command (instead of `mpiexecjl -np <np>` shown above) with:

```
$ prun -np <np> -<cores> -script $PRUN_ETC/prun-openmpi-mpiexecjl
```

The `prun-openmpi-mpiexecjl` script takes care of setting up an execution environment for Julia using the default MPI implementation (OpenMPI), before invoking the actual Julia application using `mpiexecjl`. Note that with prun on DAS-5 the number of compute nodes is specified using `-np <np>` while the number of cores per node is set by `-<cores>`; the total number of MPI processes is the multipe of these two. Between nodes, MPI employs a fast network interconnect, while within a node, it can employ some shared memory mechanism between proceses.

3.4 Correctness

The parallel program should print the full world state at the end, so the consistency of your MPI version with the original sequential version can be checked, e.g. by running `diff` on the outputs. When the simulation runs for a large number of iterations, at some point the outcome may start to diverge slightly from the sequential version. This can be the result of reordered floating point operations in the MPI version, which can result in slightly different rounding due to the nature of floating point implementation, and thus no reason for concern. However, for a smaller number of bodies and iterations, the printed world states should be identical for both versions.

3.5 Performance evaluation

During your tests, you should vary the number of bodies to achieve possibly different computation/communication levels, while the number of iterations is set large enough to get reliabile performance indicators. For example, choosing parameters such that a sequential run will take a few minutes would give a good starting point. Data distribution at the start (transfering the world state computed at rank 0 to the other ranks using MPI), and gathering and printing of the world state at the end of the process must also be implemented in your program. However, you should not take the time this takes into account when performing efficiency/speedup measurements.
In your strong scaling analysis, measure the execution time of your MPI implementation on DAS-5, using 1, 2, 4, or 8 nodes with 1, 2, 4, 8, or 16 cores per node. Make sure you use the right MPI operations for the best performance. For example, use MPI's collective communication operations whenever that is useful (e.g., to do a broadcast, use MPI's broadcast primitive, and not N serial send operations).
Some node/core combinations have the same overall number of cores (e.g., 16 MPI ranks can be obtained using 1 node with 16 cores, but also using 2 nodes with 8 cores, etc.) which can give different performance depending on what is the most limiting factor in the parallel computation (e.g., CPU cache, memory bandwidth, network overhead, etc). Look at the performance trends when scaling up compute nodes and cores, and formulate some performance hypotheses regarding the trends you observe.
You can use the timers provided by Julia to measure the performance of sections of your code, or you can use the Julia profiler interface to get a first visual impression (see the information on Canvas how to do this).

3.6 Report

Write a short report (maximum 7 pages, including graphs) describing your design, implementation, and performance evaluation for your parallel N-body solution. Describe (briefly) the way the algorithm works, focusing on the MPI implementation details.
When presenting the performance results, discuss the composition of the experimental setup - number of nodes and cores, input parameters, application versions, etc., and the way you measured the performance. Report performance by means of speedup and/or parallel efficiency graphs (typically efficiency graphs provide the most insightful presentation when scaling up the total number of cores in strong scaling experiments). Discuss the efficiency trends, compared with the ideal parallel efficiency of 1.0, and try to explain possibly unexpected results.

4 Assignment

Objectives

In order to make sure you complete this part of the practical, we recommend the following recipe:

- Check the sequential version of the Julia N-body application.
- Create a parallel Julia application using MPI based on this.
- Test the functionality and correctness of your application.
- Make sure you check the application runs correctly for various input parameters.
- Determine suitable parameters (i.e., number of bodies and number of timesteps) for your performance evaluation.
- Evaluate the sequential and parallel versions on DAS-5, using prun as described, or SLURM batch scripts if you prefer.

Write a report about your MPI assignment. Mention the difficulties you encountered and how you solved them. Also, present the experimental results and discuss the achieved performance of your solution. Include speedup and/or parallel efficiency graphs (tables with listings of individual performance numbers are not needed).
To turn in your assignment, make a .zip file with your parallel implementations and your report pdf file, and submit the archive via Canvas. You can resubmit before the deadline, and only the last submission found on the site at the submission deadline will be graded.

Grading

- 3 points: for a correct parallel implementation of the base and load balanced versions, compliant with all the requirements;
- 1 point: for coding style (use clear, commented code);
- 4 points: for the report with performance evaluation and analysis;
- 2 points: additional points for an implementation supporting the position gathering, with evaluation and analysis.

Deliverables

You have to submit both the code (containing useful comments regarding your implementation) and the report.
Important: Because we use automatic test scripts to test and benchmark your submissions, you must strictly follow the instructions below. Submissions that do not follow these instructions may receive a penalty.

- The MPI version (implementing the base version and extensions) should consist of the files `nbody-par-src.jl` and `nbody-par-run.jl` using the calling conventions discussed.
- Make sure you evaluate with `-O3` optimizations turned on, because this impacts performance.
- Make sure that your parallel program gives the same output as the sequential program. Because we compare your application's output with the correct output using `diff`, a significant difference (not counting small differences due to possible floating point operation reordering) suggests a bug in your code, which could invalidate your evaluation and analysis and thus severely impact your overall grade.
- Create the report in PDF format as a file called `report.pdf`.
- Place your MPI implementation and report in a directory called `MPI-GroupXX` (where XX is your group number) and archive this to a zip file `MPI-GroupXX.zip`. Only one of of the group members needs to submit this zip file via Canvas.

5 Contact

For any questions about this assignment, contact Kees Verstoep (`c.verstoep@vu.nl`).